Technical Case Study: Job Radar Scoring Engine

Candidate: [Taylan Özgür Özdemir] **Target Role:** Junior Software Engineer @ Sezzle **Date:** February 2026

🚀 Overview

This document showcases a refactored module of the **Job Radar** project. It focuses on the core scoring engine, demonstrating professional engineering standards including:

- **Domain Modeling** with Pydantic Enums and BaseModels.
- **Service-Oriented Architecture** for clean separation of concerns.
- **Defensive Programming** via runtime data validation.
- **Test-Driven Mentality** with comprehensive unit test coverage.

1. Domain Modeling (`models.py`)

I use **Pydantic** to enforce type safety and validate data during ingestion. This is critical for Fintech applications where data integrity is paramount.

```
from enum import Enum
from typing import Optional, List
from pydantic import BaseModel, Field, field_validator

class JobField(str, Enum):
    GAMEDEV = "gamedev"
    CSHARP = "c#"
    PYTHON = "python"
    CPP = "c++"
    MOBILE = "mobile"
    JAVA = "java"
    CV = "cv"
    BACKEND = "backend"
    FRONTEND = "frontend"
    SOFTWARE = "yazılım"
    OTHER = "diğer"

class JobLocation(str, Enum):
    ANKARA = "ankara"
    REMOTE = "remote"
    HYBRID_ANKARA = "hybrid_ankara"
    TURKIYE = "türkiye"
    ISTANBUL = "istanbul"
    IZMIR = "izmir"
    BURSA = "bursa"
    EUROPE = "avrupa"
    USA = "abd"
    OTHER = "diğer"
```

```
    DEAD = "dead"

class JobLevel(str, Enum):
    JUNIOR = "junior"
    ENTRY = "entry"
    MID = "mid"
    SENIOR = "senior"
    LEAD = "lead"
    UNKNOWN = "bilinmiyor"

class Job(BaseModel):
    """Represents a raw job listing before processing."""
    title: str
    company: str
    location: str
    summary: str = ""
    description: Optional[str] = ""
    link: Optional[str] = None
    published: Optional[str] = None

    @field_validator('title', 'company')
    @classmethod
    def not_empty(cls, v: str) -> str:
        if not v.strip():
            raise ValueError('Must not be empty')
        return v.strip()

class ScoredJob(Job):
    """Represents a job listing after being processed and scored."""
    detected_field: JobField
    detected_location: JobLocation
    detected_level: JobLevel
    total_score: float
    is_restricted: bool = False
    restricted_reason: Optional[str] = None
```

---

2. Business Logic (`scoring_service.py`)

The `ScoringService` encapsulates the logic for job analysis. It uses a hybrid approach (NLP + Patterns) and follows the **Dependency Injection** principle.

```
import re
import logging
from typing import Optional, Tuple

from models import Job, ScoredJob, JobField, JobLocation, JobLevel
from nlp_processor import normalize_text, detect_language, FieldDetector
import config

logger = logging.getLogger(__name__)
```

```python
class ScoringService:
    """
    Professional service for analyzing, categorizing, and scoring job listings.
    """

    def __init__(self, cfg=config):
        self.cfg = cfg
        self._field_detector: Optional[FieldDetector] = None

    @property
    def field_detector(self) -> FieldDetector:
        if self._field_detector is None:
            self._field_detector = FieldDetector()
        return self._field_detector

    def process_job(self, job: Job) -> ScoredJob:
        text = f"{job.title} {job.summary} {job.description} {job.location}
{job.company}"
        normalized_text = normalize_text(text)

        # 1. Restriction & Language Check
        is_restricted, reason = self._check_restrictions(normalized_text)

        # 2. Categorization
        field = self._detect_field(normalized_text)
        location = self._detect_location(normalized_text)
        level = self._detect_level(normalized_text)

        if reason and "Language" in reason:
            location = JobLocation.DEAD

        # 3. Scoring
        total_score = -10.0 if is_restricted else self._calculate_sum(field,
location, level)

        return ScoredJob(
            **job.model_dump(),
            detected_field=field,
            detected_location=location,
            detected_level=level,
            total_score=total_score,
            is_restricted=is_restricted,
            restricted_reason=reason
        )

    def _check_restrictions(self, text: str) -> Tuple[bool, Optional[str]]:
        for kw in self.cfg.NEGATIVE_KEYWORDS:
            if kw.lower() in text:
                return True, f"Restriction found: {kw}"

        lang = detect_language(text)
        if lang and lang not in ["en", "tr"]:
            return True, f"Unsupported Language: {lang}"
```

```
        return False, None

    def _detect_field(self, text: str) -> JobField:
        job_title_lower = text[:100]

        # Fast Path Overrides
        if re.search(r'\b(game dev|game design|game program|unity|unreal)\w*',
text):
            return JobField.GAMEDEV

        if any(kw in text for kw in ["frontend", "front-end"]):
            return JobField.FRONTEND

        if any(kw in text for kw in ["backend", "back-end"]):
            return JobField.BACKEND

        # NLP Context Inference
        try:
            detected = self.field_detector.predict(text)
            if detected:
                # Handle potential NLP bias for generic titles
                is_generic = any(kw in job_title_lower for kw in ["software
engineer", "developer"])
                if detected == "c#" and is_generic:
                    if self._count_hits(text, self.cfg.FIELD_KEYWORDS.get("c#",
[])) == 0:
                        detected = None

                if detected: return JobField(detected)
        except Exception as e:
            logger.error(f"NLP Detection failed: {e}")

        return JobField.OTHER

    def _detect_location(self, text: str) -> JobLocation:
        if "ankara" in text:
            return JobLocation.ANKARA
        if any(kw.lower() in text for kw in
self.cfg.LOCATION_KEYWORDS.get("remote", [])):
            return JobLocation.REMOTE
        return JobLocation.OTHER

    def _detect_level(self, text: str) -> JobLevel:
        junior_patterns = [r'\bjunior\b', r'\bjr\b', r'\bentry level\b', r'yeni
mezun']
        if any(re.search(p, text) for p in junior_patterns):
            return JobLevel.JUNIOR

        year_match = re.search(r'(\d+)\s*\+\s*years?', text)
        if year_match:
            min_years = int(year_match.group(1))
            if min_years >= 5: return JobLevel.SENIOR
            if min_years >= 2: return JobLevel.MID
```

```
            return JobLevel.JUNIOR

        return JobLevel.UNKNOWN

    def _calculate_sum(self, field: JobField, location: JobLocation, level:
JobLevel) -> float:
        f_score = self.cfg.FIELD_SCORES.get(field.value, 1)
        l_score = self.cfg.LOCATION_SCORES.get(location.value, 0)
        sen_score = self.cfg.LEVEL_SCORES.get(level.value, 0)

        total = (f_score * 0.3 + l_score * 0.4 + sen_score * 0.3)
        return round(total, 2)

    def _count_hits(self, text: str, keywords: list) -> int:
        return sum(1 for kw in keywords if kw.lower() in text)
```

---

3. Quality Assurance (`tests/test_scoring_service.py`)

Reliability is proven through automated unit tests. I use **Pytest** to validate various scenarios and edge cases.

```
import pytest
from models import Job, JobField, JobLocation, JobLevel, ScoredJob
from scoring_service import ScoringService

@pytest.fixture
def service():
    return ScoringService()

def test_process_job_standard_case(service):
    """Tests correct categorization and scoring for a Fintech role."""
    job = Job(
        title="Software Engineer",
        company="Sezzle",
        location="Remote",
        summary="Looking for a Junior Backend role.",
        description="Stack includes Python, FastAPI, and AWS."
    )
    result = service.process_job(job)

    assert result.detected_field == JobField.BACKEND
    assert result.detected_location == JobLocation.REMOTE
    assert result.detected_level == JobLevel.JUNIOR
    assert result.total_score > 0

def test_process_job_restricted(service):
    """Ensures jobs with specific restrictions are flagged and zero-scored."""
    job = Job(
        title="Engineer",
        company="US Tech",
        location="USA",
```

```
            summary="US Citizenship required."
        )
        result = service.process_job(job)
        assert result.is_restricted is True
        assert result.total_score == -10.0

    def test_seniority_regex(service):
        """Validates year-based seniority detection logic."""
        job = Job(title="Dev", company="A", location="L", summary="5+ years
    experience")
        assert service.process_job(job).detected_level == JobLevel.SENIOR
```

Summary

This showcase demonstrates how I apply **SOLID principles** and **modern Python patterns** to build real-world tools. My goal is to write code that is not only functional but also maintainable, scalable, and fully tested.